

Decision Trees & Random Forests.

Decision Tree

- A tree-like model that illustrates series of events leading to certain decisions
- Each node represents a test on an attribute and each branch is an outcome of that test

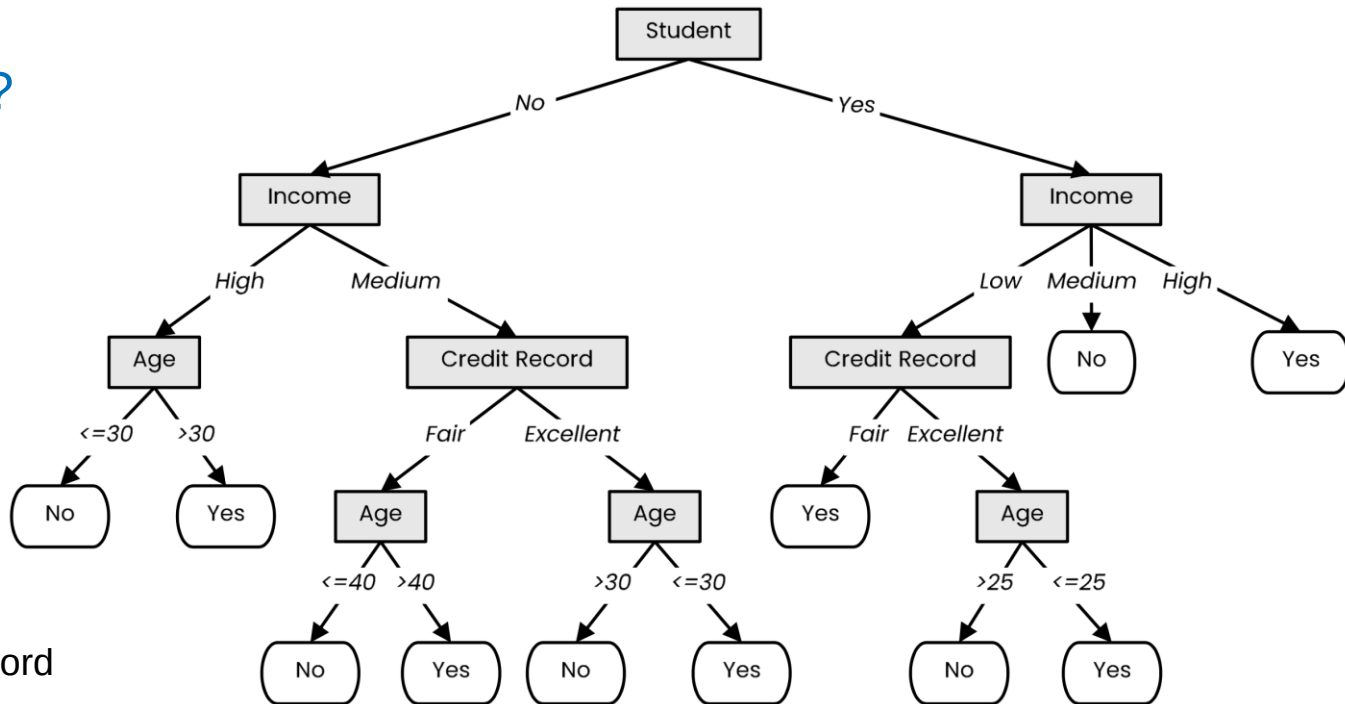
Whom to loan?



- Not a student
- 45 years old
- Medium income
- Fair credit record



- Student
- 27 years old
- Low income
- Excellent credit record



Definition

- A tree-like model that illustrates series of events leading to certain decisions
- Each node represents a test on an attribute and each branch is an outcome of that test

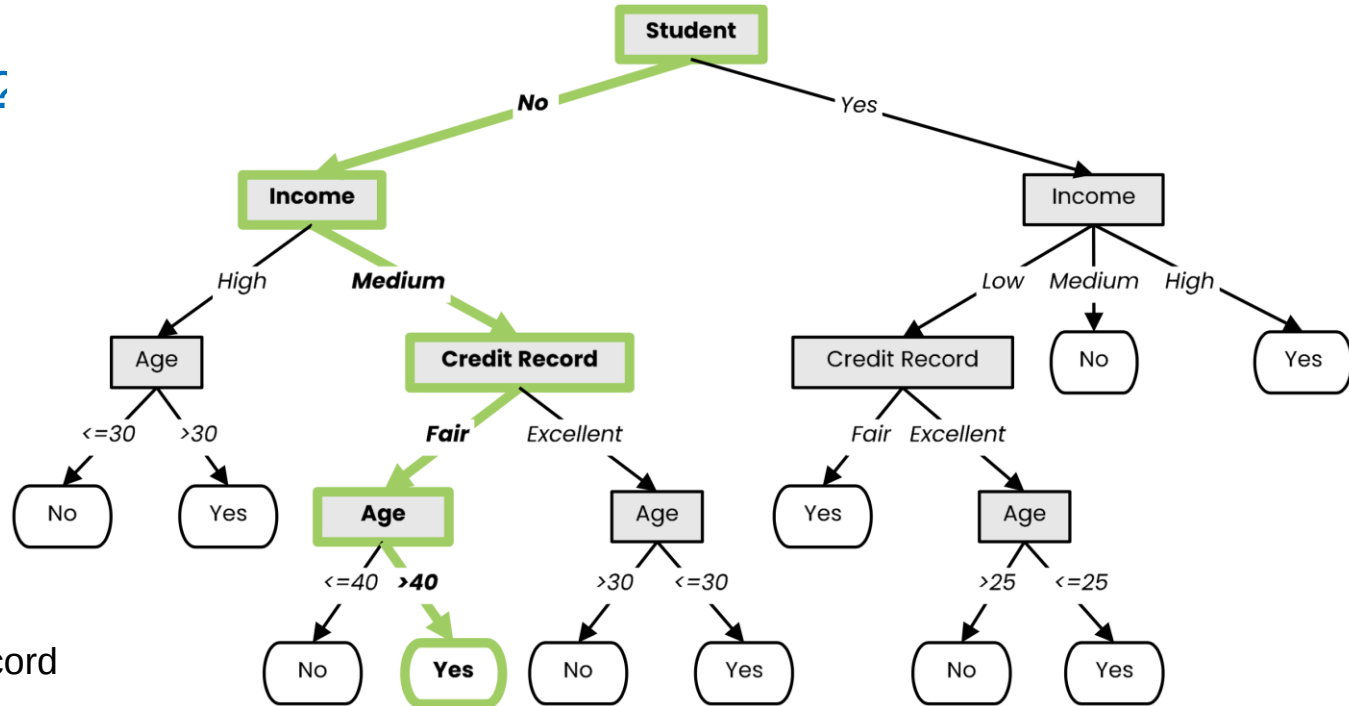
Whom to loan?



- Not a student
 - 45 years old
 - Medium income
 - Fair credit record
- Yes



- Student
- 27 years old
- Low income
- Excellent credit record



Definition

- A tree-like model that illustrates series of events leading to certain decisions
- Each node represents a test on an attribute and each branch is an outcome of that test

Whom to loan?



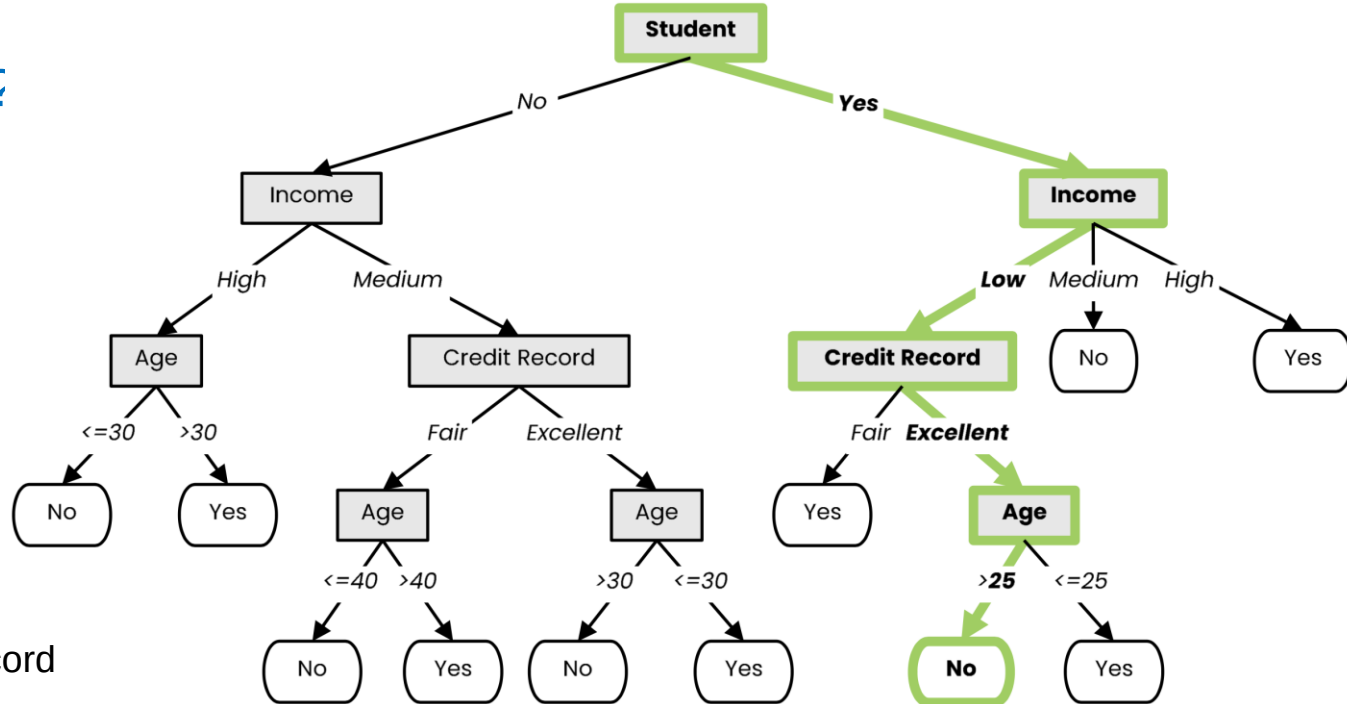
- Not a student
- 45 years old
- Medium income
- Fair credit record

➤ Yes



- Student
- 27 years old
- Low income
- Excellent credit record

➤ No



Pruning

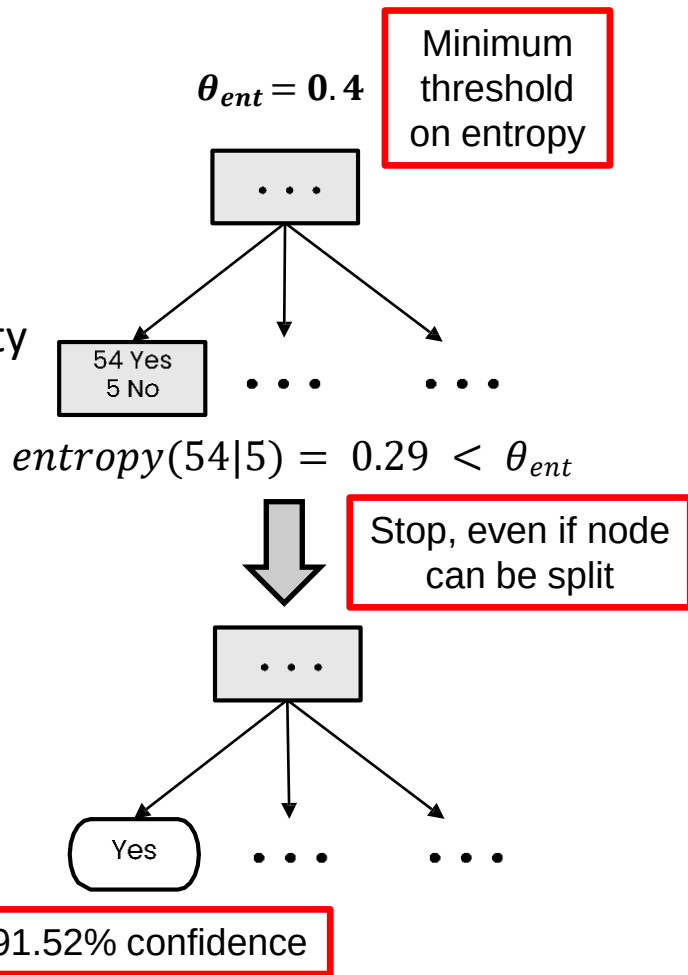
- Pruning is a technique that reduces the size of a decision tree by removing branches of the tree which provide little predictive power
- It is a **regularization** method that reduces the complexity of the final model, thus reducing overfitting
 - Decision trees are prone to overfitting!
- Pruning methods:
 - Pre-pruning: Stop the tree building algorithm before it fully classifies the data
 - Post-pruning: Build the complete tree, then replace some non-leaf nodes with leaf nodes if this improves validation error

Pre-pruning

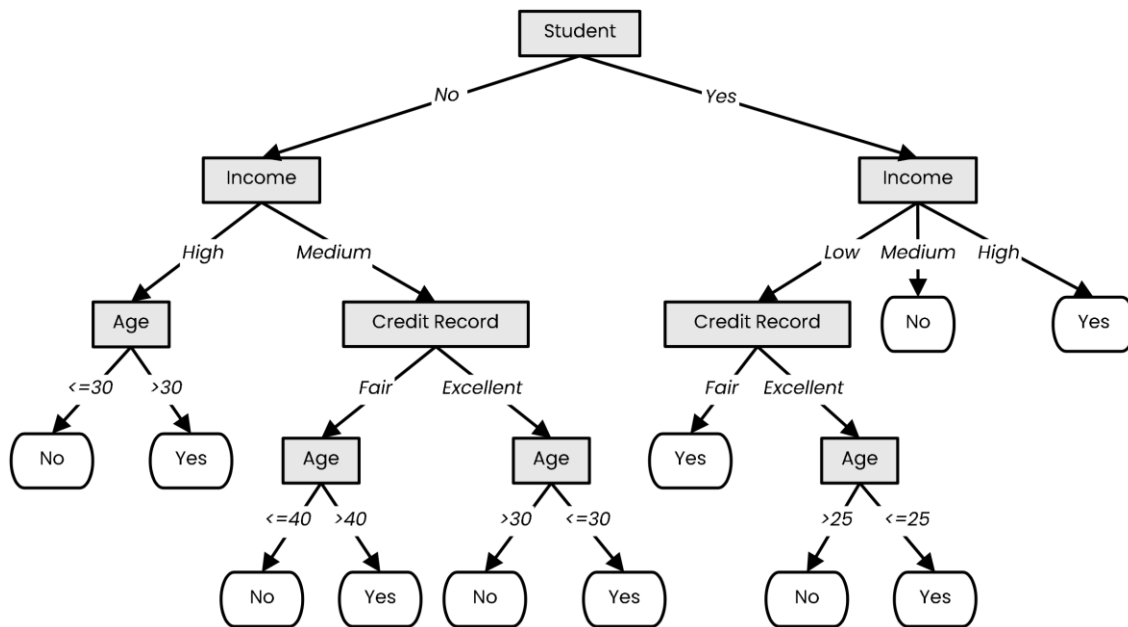
- Pre-pruning implies early stopping:
- If some condition is met, the current node will not be split, even if it is not 100% pure
- It will become a leaf node with the label of the majority class in the current set
- (the class distribution could be used as prediction confidence)

- **Common stopping criteria include setting a threshold on:**

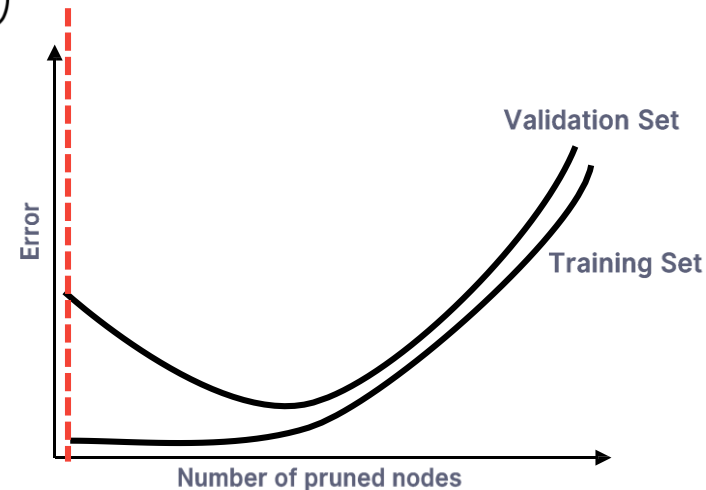
- Entropy (or Gini Impurity) of the current set
- Number of samples in the current set
- Gain of the best-splitting attribute
- Depth of the tree



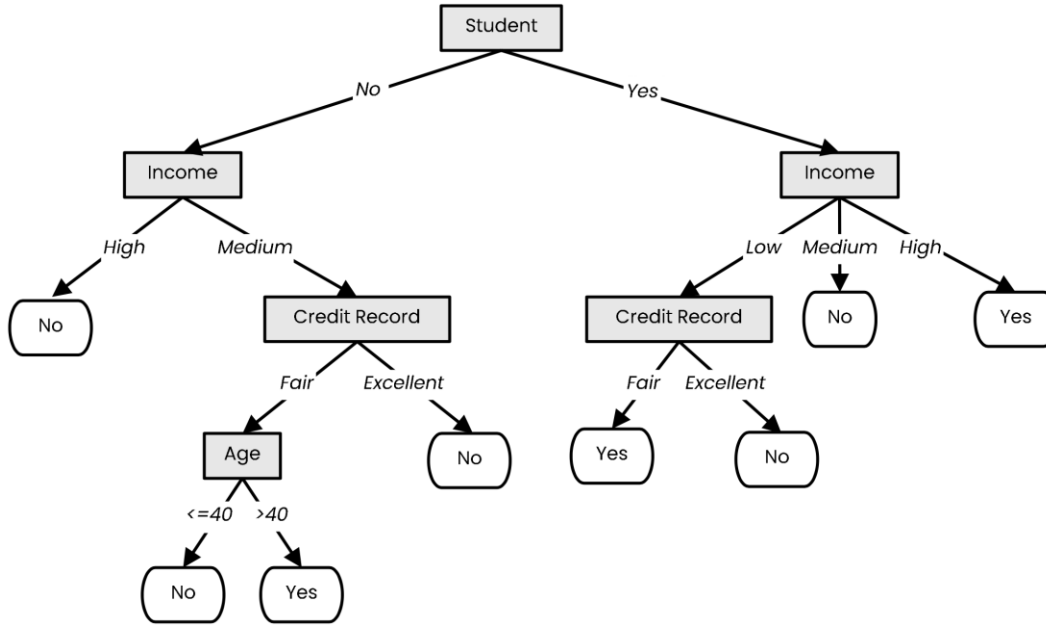
Post-pruning



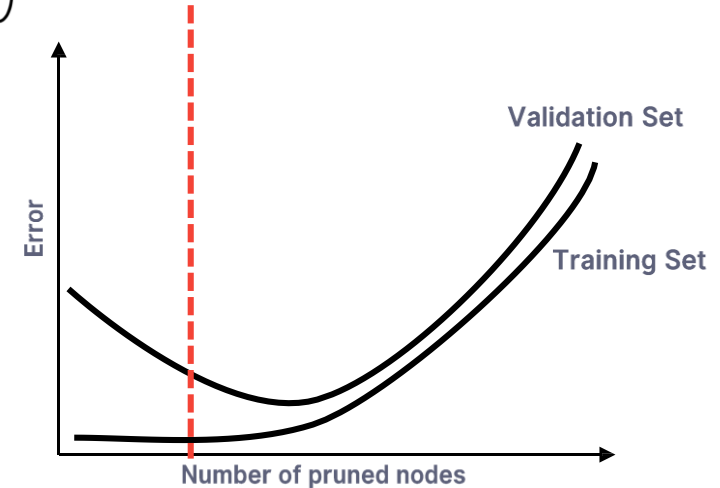
- Prune nodes in a bottom-up manner, if it decreases validation error



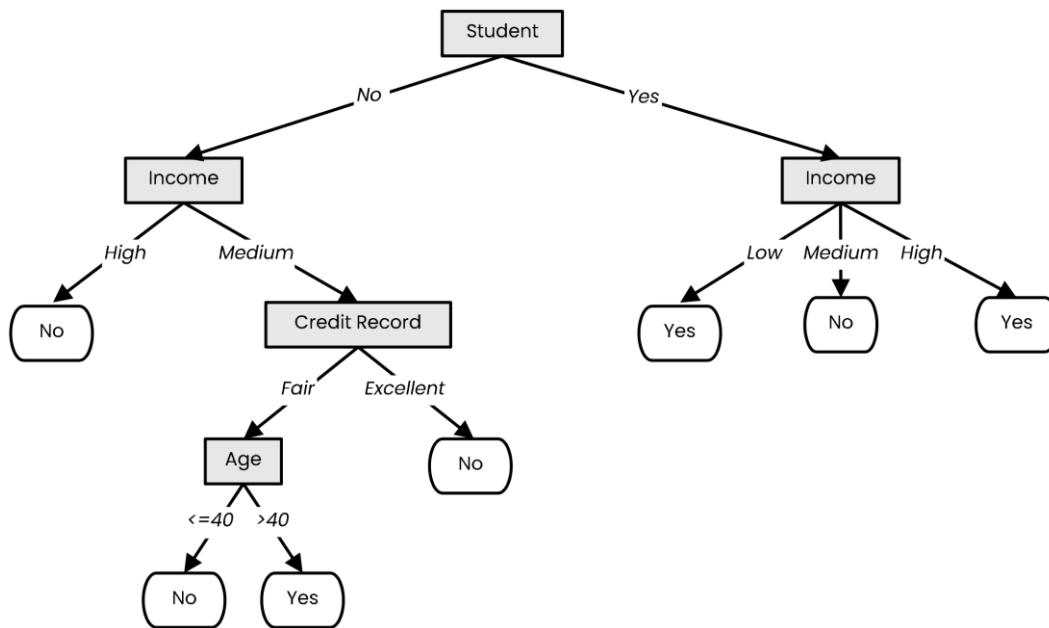
Post-pruning



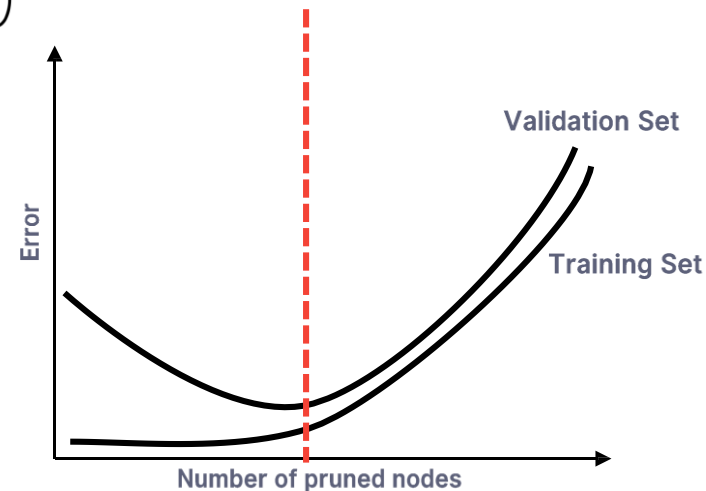
- Prune nodes in a bottom-up manner, if it decreases validation error



Post-pruning



- Prune nodes in a bottom-up manner, if it decreases validation error



Summary

- Decision trees represent a tool based on a tree-like graph of decisions and their possible outcomes
- Decision tree learning is a machine learning method that employs a decision tree as a predictive model
- ID3 builds a decision tree by iteratively splitting the data based on the values of an attribute with the largest information gain (decrease in entropy)
 - Using the decrease of Gini Impurity is also a commonly-used option in practice
- C4.5 is an extension of ID3 that handles attributes with continuous values, missing values and adds regularization by pruning branches likely to overfit

Ensemble Learning

- Ensemble Learning:
 - Method that combines multiple learning algorithms to obtain performance improvements over its components
- Commonly-used ensemble methods:
 - **Random Forests** are one of the most common examples of ensemble learning
 - **Bagging**: multiple models on random subsets of data samples
 - **Random Subspace Method**: multiple models on random subsets of features
 - **Boosting**: train models iteratively, while making the current model focus on the mistakes of the previous ones by increasing the weight of misclassified samples

Boosting

- **Boosting** is an **ensemble learning technique** where multiple **weak learners** (usually simple models like shallow decision trees) are trained **sequentially**, and each new model focuses on correcting the errors made by the previous ones.
- The main idea is to convert many weak models into a **strong predictive model**.

Working of Boosting

1. Train the first weak learner

1. A simple model (e.g., a small decision tree) is trained on the dataset.
2. All training samples initially have **equal importance (weights)**.

2. Evaluate errors

1. After prediction, the algorithm identifies which samples are **misclassified**.

3. Increase importance of difficult (miss classified) samples

1. The misclassified samples are assigned **higher weights**.
2. Correctly classified samples receive **lower weights**.

4. Train the next weak learner

1. The next model is trained using the **updated weights**.
2. Because misclassified samples have higher weights, the new model focuses more on them.

5. Repeat the process

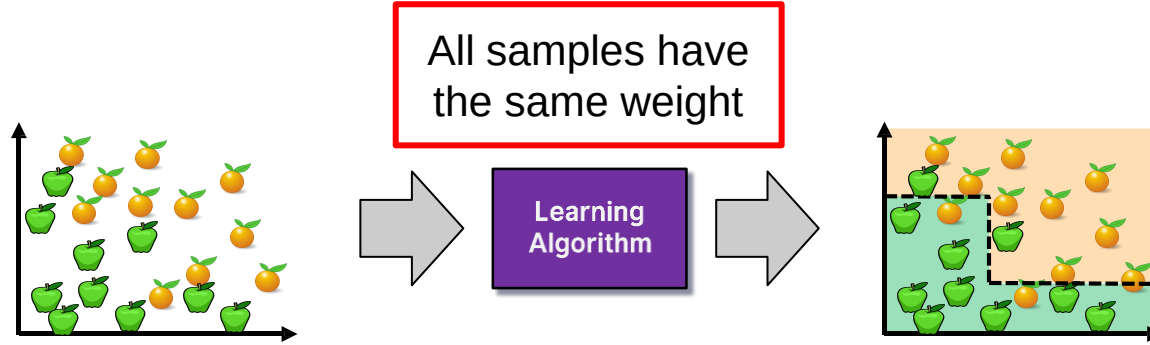
1. Multiple weak learners are trained sequentially.
2. Each new learner attempts to **correct the mistakes of the previous learners**.

6. Combine all models

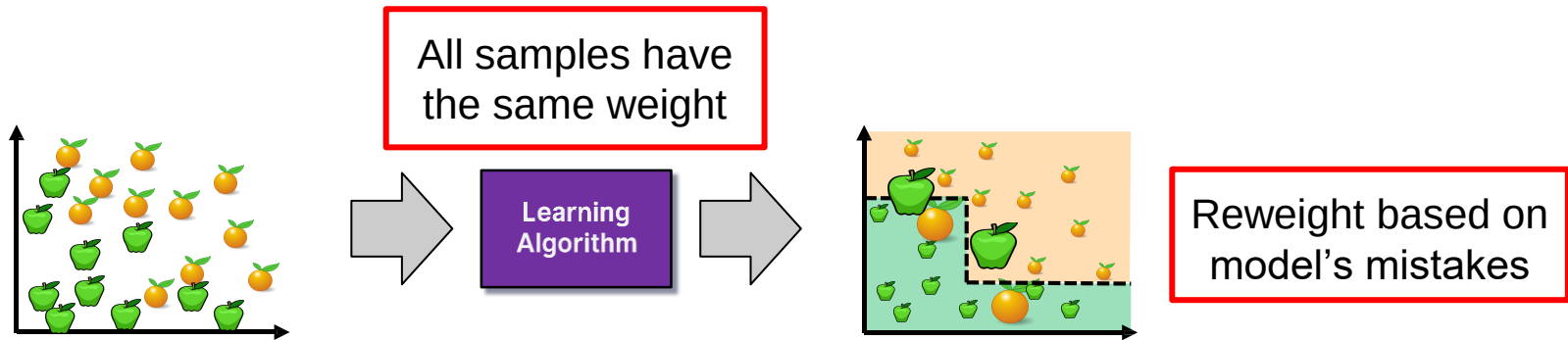
1. The predictions of all weak learners are **combined (weighted voting or summation)** to produce the final prediction.

- Note:
- In boosting with decision trees, the weights are not stored inside the tree structure itself. Instead, the training algorithm keeps a weight for each training sample, and these weights are used while building the next tree. The decision tree algorithm is modified to consider sample weights when calculating splits and errors.

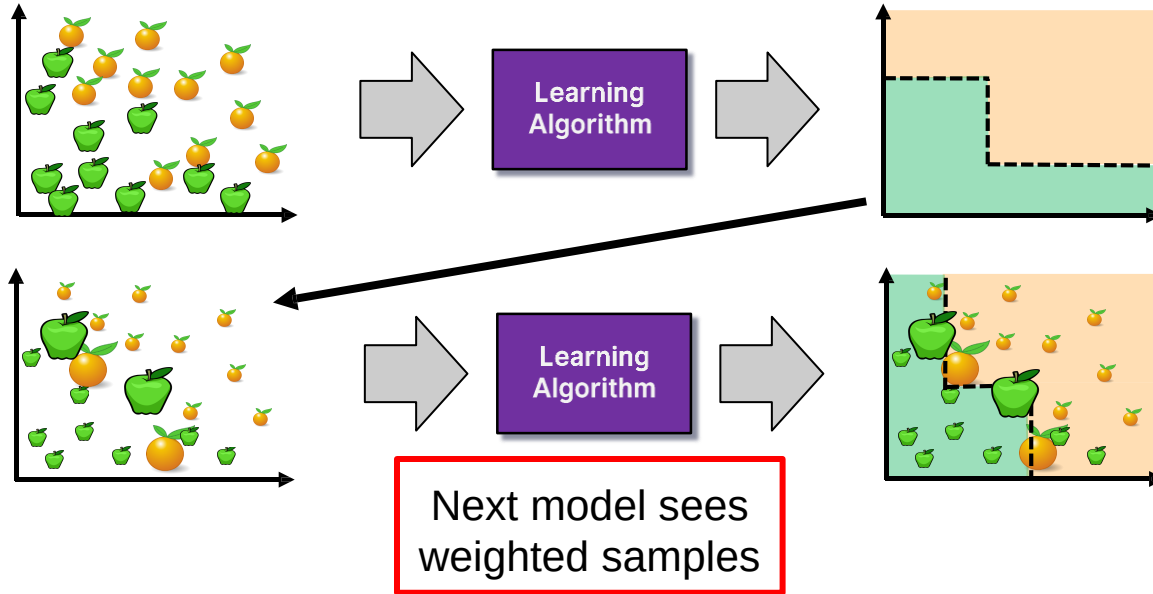
Boosting



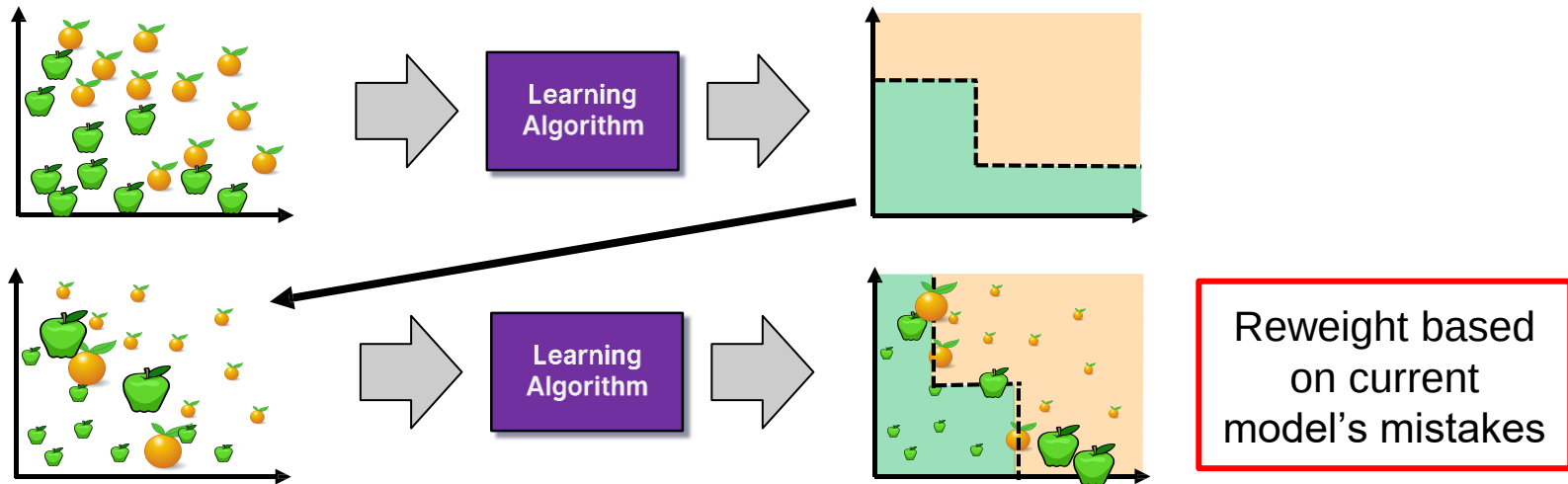
Boosting



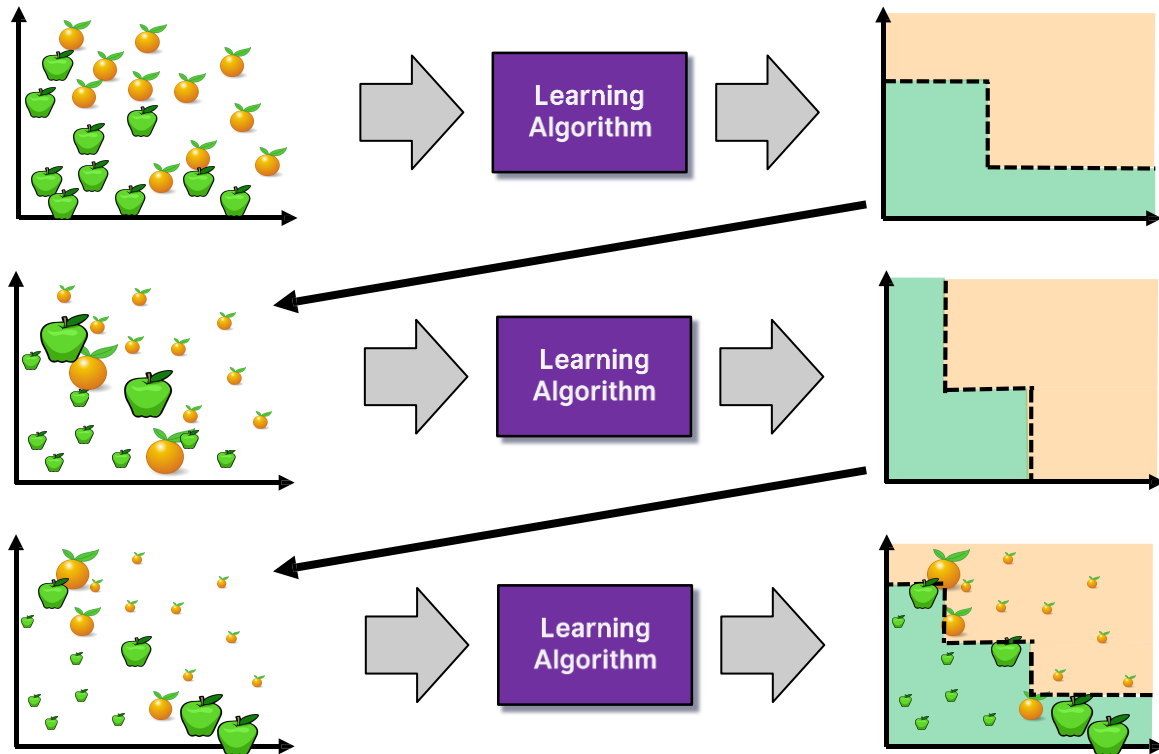
Boosting



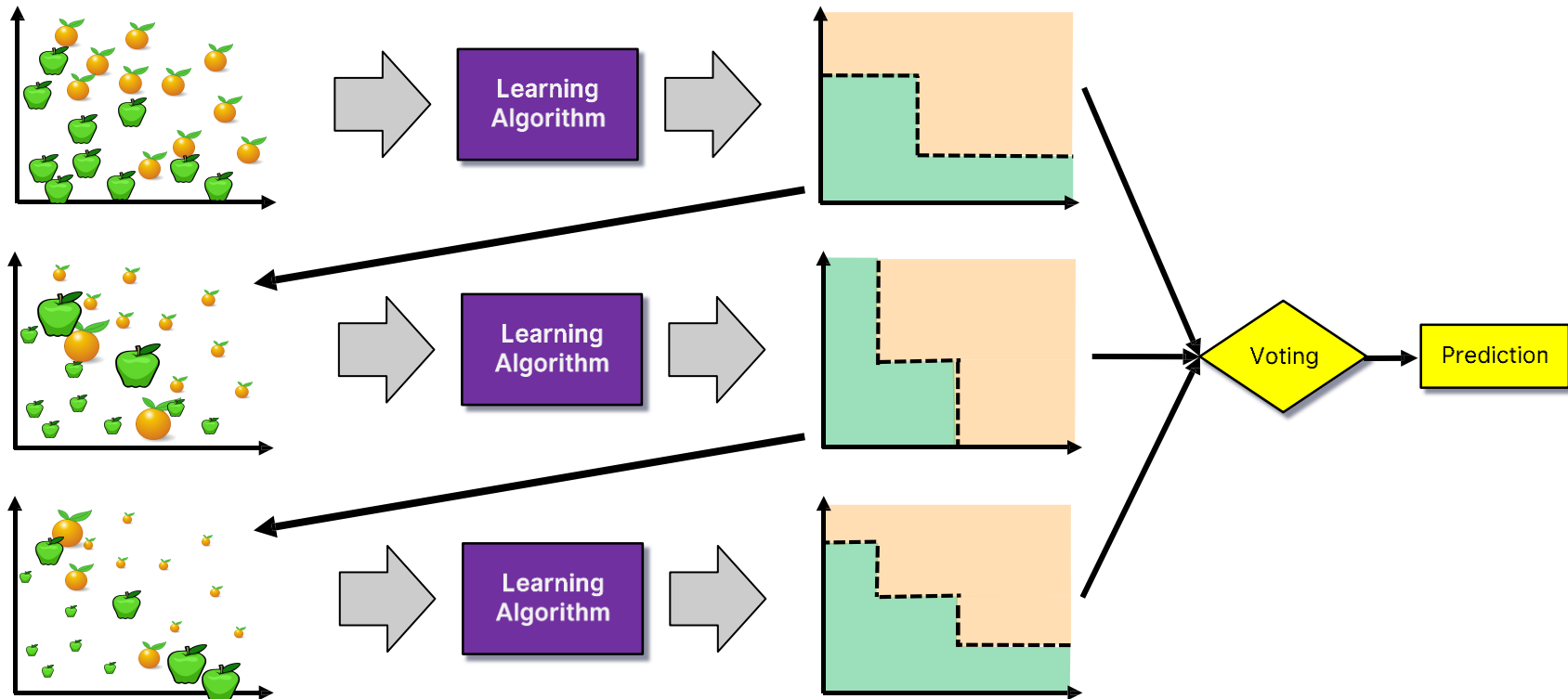
Boosting



Boosting



Boosting



Random Forests

(Ensemble learning with decision trees)

Random Forests

- Random Forests:
 - Random Forests are an ensemble learning method that employ decision tree learning to build multiple trees through bagging and random subspace method.
 - They rectify the overfitting problem of decision trees!
 - Instead of building a single decision tree and use it to make predictions, build many slightly different trees and combine their predictions
- We have a single data set, so how do we obtain slightly different trees?
 1. Bagging (**Bootstrap Aggregating**):
 - Take random subsets of data points from the training set to create N smaller data sets
 - Fit a decision tree on each subset
 2. Random Subspace Method (also known as Feature Bagging):
 - Fit N different decision trees by constraining each one to operate on a random subset of features

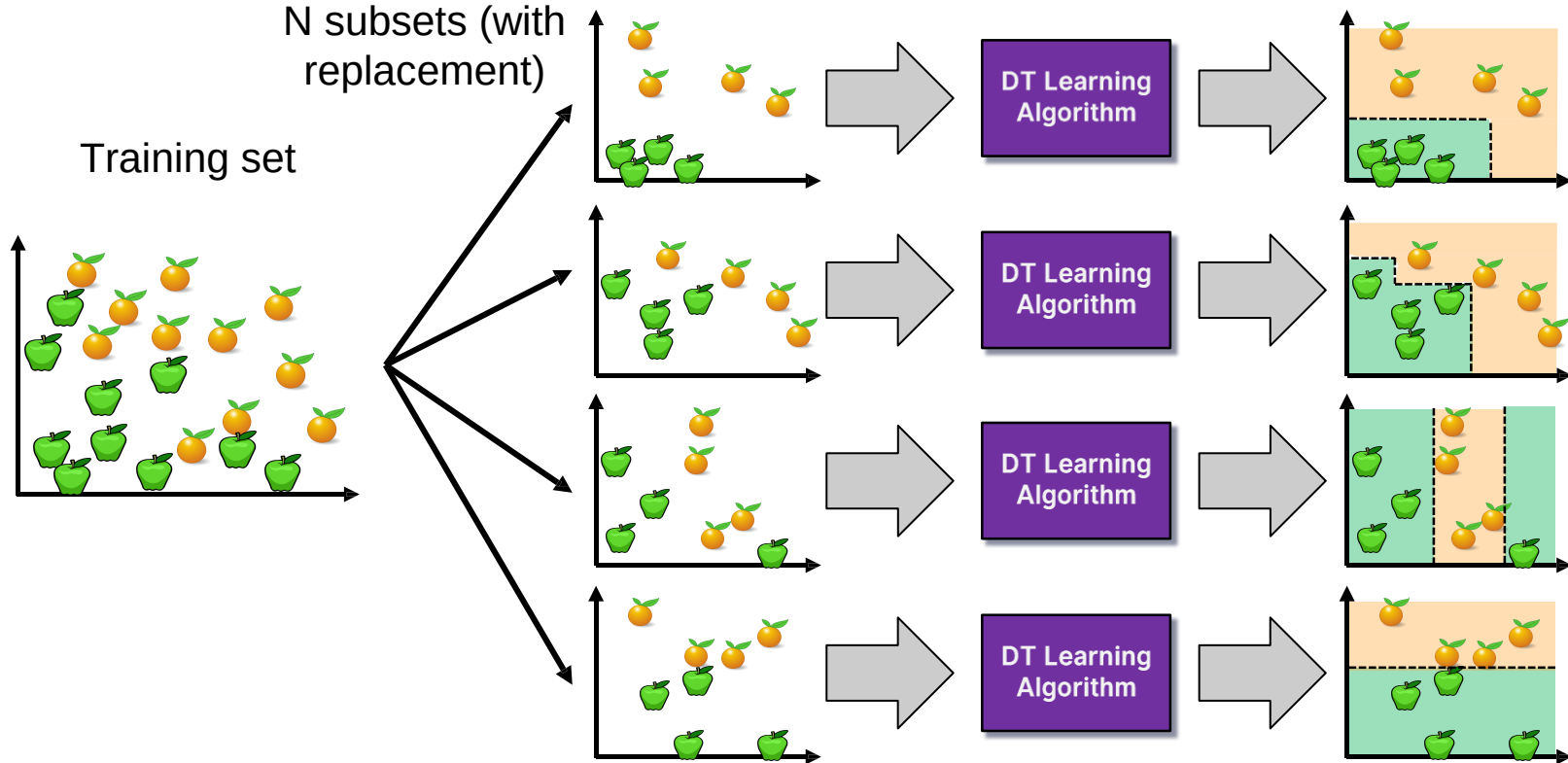
1. Bagging

- Bagging, short for Bootstrap Aggregating, is an ensemble machine learning technique designed to improve the stability and accuracy of machine learning algorithms.
- It helps reduce variance (overfitting) and works particularly well with complex models like decision trees.
- **How Bagging Works?**
- Bagging operates through a three-step process:
 1. Bootstrap Sampling: Multiple subsets of data (called bootstrap samples) are created by randomly sampling the original dataset with replacement. This means some data points may appear multiple times in a single subset, while others might be left out.
 2. Parallel Training: A base model (often called a "weak learner," such as a decision tree) is trained independently on each bootstrap subset in parallel.
 3. Aggregation: The predictions of all the individual models are combined to produce a final output:

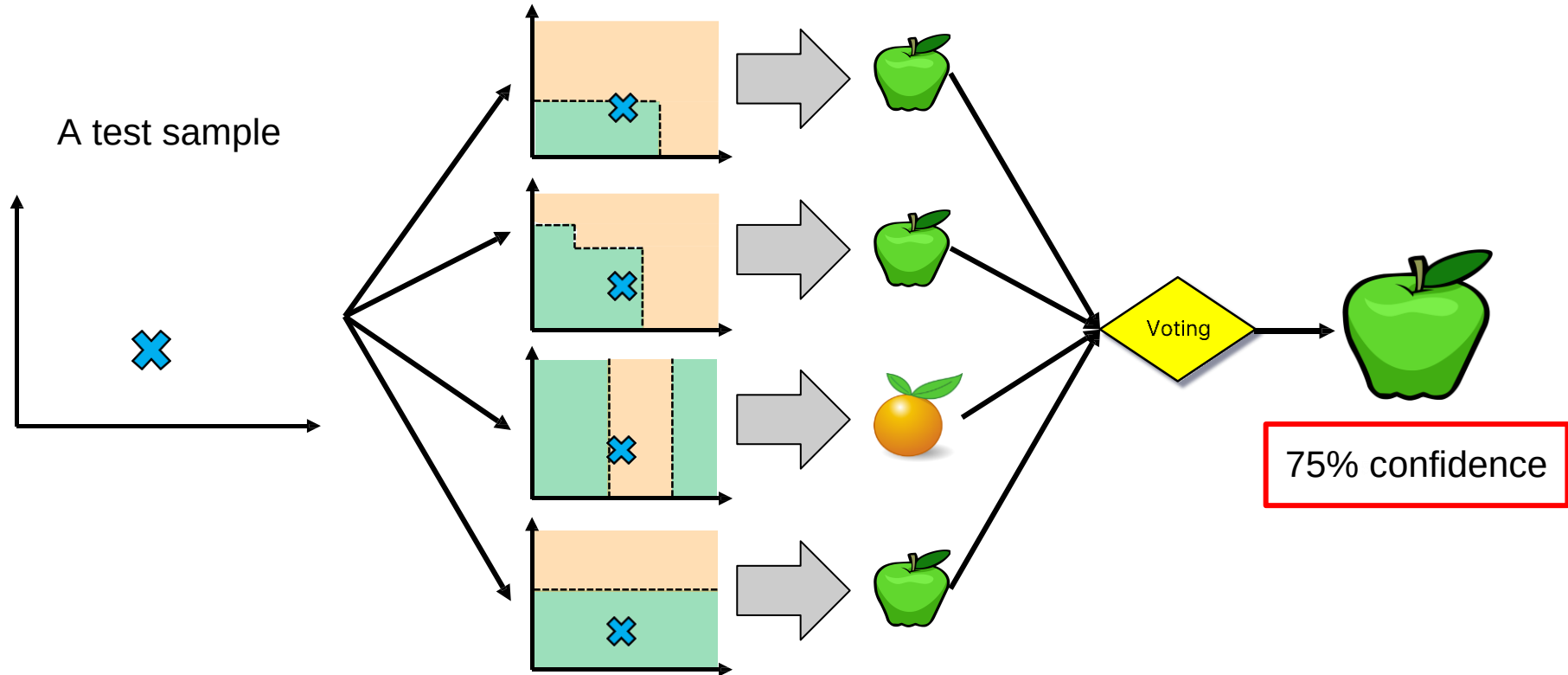
For classification tasks, this is usually done via majority voting.

For regression tasks, it is typically done by averaging the predicted values.

Bagging at training time



Bagging at inference time



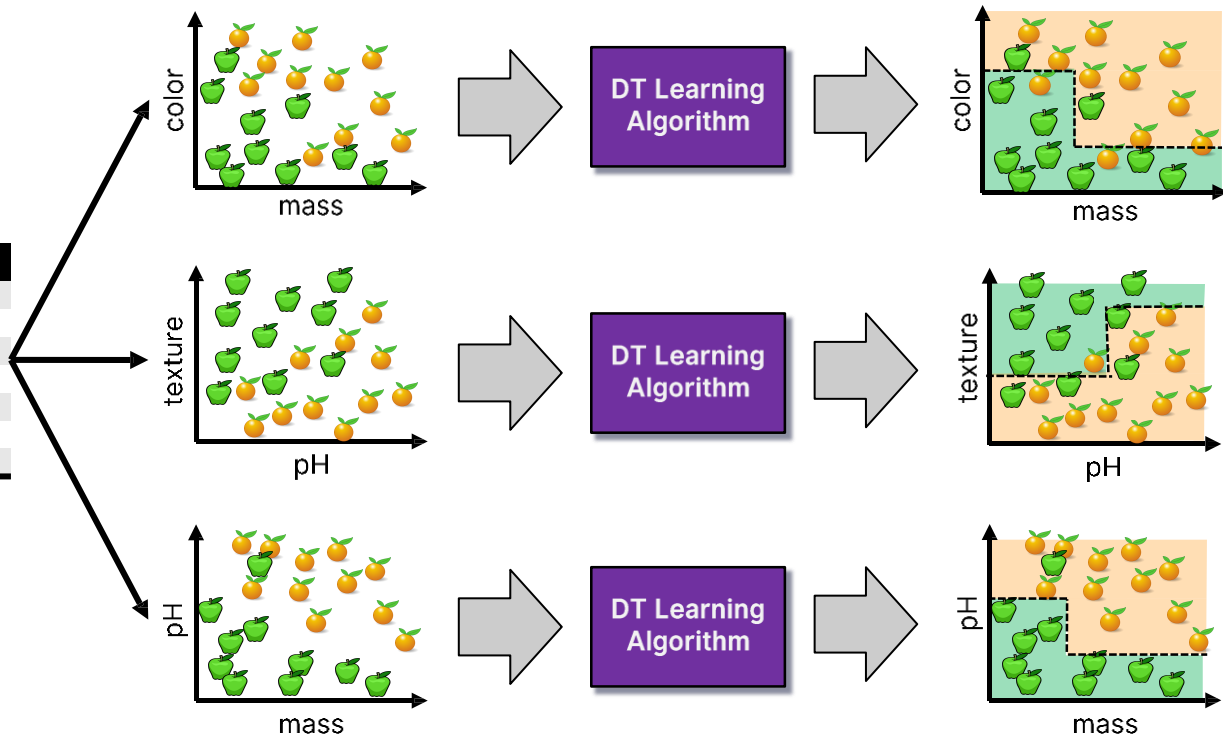
Random Subspace Method

- The Random Subspace Method, also known as feature bagging or attribute bagging, is an ensemble machine learning technique where multiple models are trained using random subsets of features (variables or columns) rather than the entire feature set.
- **How It Works:**
- Instead of feeding all available features to every model, the method operates through a targeted feature-selection process:
 1. Feature Sampling: For each base model in the ensemble, a random subset of features is selected from the total pool of available features.
 2. Independent Training: A base learner (typically a decision tree) is trained exclusively on its assigned feature subspace.
 3. Aggregation: Final predictions are combined by taking a majority vote for classification tasks or an average for regression tasks.
- **Relationship with Bagging and Random Forests**
- While traditional bagging (bootstrap aggregating) creates ensembles by sampling random subsets of data rows (observations), the Random Subspace Method samples random subsets of data columns (features).
- In modern machine learning, these concepts are often combined. For instance, the Random Forest algorithm uses both techniques simultaneously: it applies row bagging (bootstrap samples) and the random subspace method (random feature selection at each tree split).

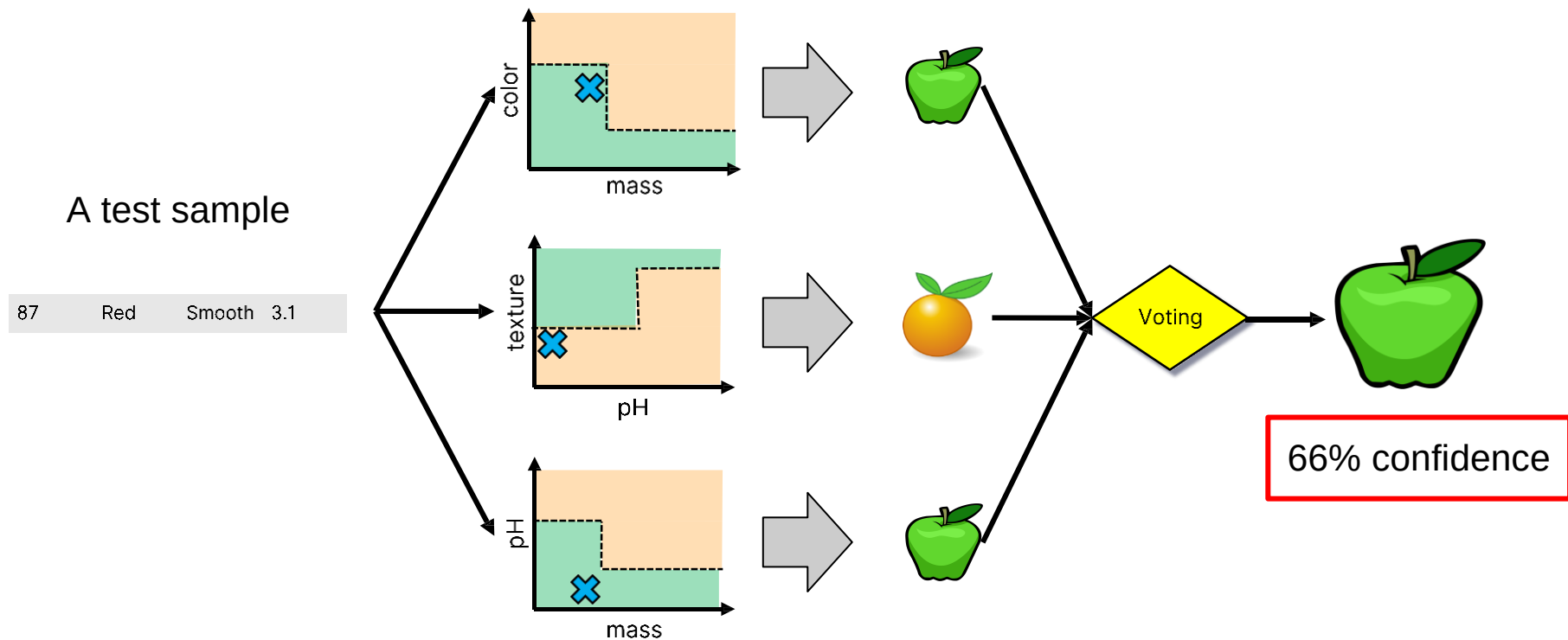
Random Subspace Method at training time

Training data

Mass (g)	Color	Texture	pH	Label
84	Green	Smooth	3.5	Apple
121	Orange	Rough	3.9	Orange
85	Red	Smooth	3.3	Apple
101	Orange	Smooth	3.7	Orange
111	Green	Rough	3.5	Apple
...				
117	Red	Rough	3.4	Orange



Random Subspace Method at inference time



Random Forests

Mass (g)	Color	Texture	pH	Label
84	Green	Smooth	3.5	Apple
121	Orange	Rough	3.9	Orange
85	Red	Smooth	3.3	Apple
101	Orange	Smooth	3.7	Orange
111	Green	Rough	3.5	Apple
...				
117	Red	Rough	3.4	Orange



Bagging +
Random Subspace Method +
Decision Tree Learning Algorithm



Decision Trees and Random Forest (Python)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

clf = DecisionTreeClassifier(criterion = "entropy", min_samples_leaf = 3)
# Lots of parameters: criterion = "gini" / "entropy";
#                       max_depth;
#                       min_impurity_split;

clf.fit(X, y) # It can only handle numerical attributes!
# Categorical attributes need to be encoded, see LabelEncoder and OneHotEncoder

clf.predict([x]) # Predict class for x

clf.feature_importances_ # Importance of each feature
clf.tree_ # The underlying tree object

clf = RandomForestClassifier(n_estimators = 20) # Random Forest with 20 trees
```